

Reviewer Pack — Minimal Rank of Configuration Spaces over Boolean Functions ...

Entry b63cc36f09d9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 12:50:26 UTC

Minimal Rank of Configuration Spaces over Boolean Functions vs AC^0 Circuit Depth

Entry ID: b63cc36f09d9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 12:50:26 UTC

1. Conjecture

Field A (mathematical branch): Configuration Space Theory **Field B** (complexity object): Complexity Theory: AC^0 Circuit Complexity

Statement:

{‘text’: ‘For every read-once branching program P computing a language in AC^0 , the minimal local index of its associated configuration space is $\Theta(\log \text{size}(P))$. For languages not in AC^0 , the minimal local index grows linearly with n .’, ‘quantitative_relation’: ‘ $E[\text{min_local_index}(CS(P))] = \Theta(\log \text{size}(P))$ for $P \in AC^0$; $E[\text{min_local_index}(CS(P))] = \Theta(n)$ for $P \notin AC^0$ ’}}

Rationale (proposer’s reasoning):

{‘text’: ‘Configuration spaces are a geometric tool that can capture the complexity of combinatorial structures. By linking their minimal local index to AC^0 circuit depth, we may uncover new insights into the structure of read-once branching programs and potentially lead to improved lower bounds.’, ‘explanation’: ‘The configuration space provides a way to visualize and study complex structures. If the conjecture holds, it would suggest that the complexity of an algorithm can be reflected in the geometry of its configuration space.’}}

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 514711f3e29e3e7f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for read-once branching programs P of size $n \leq 40$ in AC^0 , the average minimal local index $E[\text{min_local_index}(CS(P))]$ is within $\Theta(\log \text{size}(P))$ ($\pm 10\%$ tolerance). For languages not in AC^0 , $E[\text{min_local_index}(CS(P))]$ should be within $\Theta(n)$ ($\pm 5\%$ tolerance).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "configuration space theory" AND "AC⁰ circuit complexity" AND minimal rank" - "local index" IN CONFIGURATION SPACE Theory AND AC⁰ Circuit Depth" - "minimal local index" vs. "AC⁰ circuit depth"

Top relevant hits considered: - [http://arxiv.org/abs/0804.4433v1] SPACE: the SPectroscopic All-sky Cosmic Explorer - [http://arxiv.org/abs/1910.01506v1] Whistler instability stimulated by the suprathermal electrons present in space plasmas - [http://arxiv.org/abs/1804.02985v1] Verification of Space Weather Forecasts issued by the Met Office Space Weather Operations Centre - [http://arxiv.org/abs/2105.10886v1] VS-Net: Voting with Segmentation for Visual Localization - [http://arxiv.org/abs/2602.00982v1] Navigating Simply, Aligning Deeply: Winning Solutions for Mouse vs. AI 2025 - [http://arxiv.org/abs/2504.07569v2] Wide Binaries from Gaia DR3 : testing GR vs MOND with realistic triple modelling

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_branching_program(n):
    program = []
    for _ in range(n):
        if random.choice([True, False]):
            program.append(0) # Left child
        else:
            program.append(1) # Right child
    return program

def construct_configuration_space(program):
    n = len(program)
    CS = [[0] * (n + 1) for _ in range(n + 1)]
    CS[0][0] = 1

    for i in range(1, n + 1):
        for j in range(i + 1):
            if program[i - 1] == 0:
                CS[i][j] += CS[i - 1][j]
```

```

        else:
            CS[i][j] += CS[i - 1][j - 1]

    return CS

def min_local_index(CS):
    n = len(CS) - 1
    min_index = float('inf')
    for i in range(n + 1):
        if CS[n][i] > 0:
            min_index = min(min_index, math.log2(i + 1))
    return min_index

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5): # Test each n with 5 different programs
            program = generate_branching_program(n)
            CS = construct_configuration_space(program)
            min_index = min_local_index(CS)
            total_metric_value += min_index
            instances_tested += 1

            if conjecture_holds:
                if n in [5, 10, 15, 20]:
                    expected_min_index = math.log2(n + 1)
                    if abs(min_index - expected_min_index) > 0.1 * expected_min_index:
                        conjecture_holds = False
                        counterexample = f"n={n}, min_local_index={min_index}"
                else:
                    expected_min_index = n
                    if abs(min_index - expected_min_index) > 0.05 * n:
                        conjecture_holds = False
                        counterexample = f"n={n}, min_local_index={min_index}"

    return {
        "metric_name": "min_local_index",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 73)) # Default to first 30 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

: 'min_local_index', 'metric_value': 3.189157451286596, 'instances_tested': 30, 'conjecture_holds': 30
TRIAL: {'metric_name': 'min_local_index', 'metric_value': 3.1758956021163027, 'instances_tested': 30, 'conjecture_holds': 30}
TRIAL: {'metric_name': 'min_local_index', 'metric_value': 3.198561244460264, 'instances_tested': 30, 'conjecture_holds': 30}
TRIAL: {'metric_name': 'min_local_index', 'metric_value': 3.015777896727958, 'instances_tested': 30, 'conjecture_holds': 30}
TRIAL: {'metric_name': 'min_local_index', 'metric_value': 3.173452853154023, 'instances_tested': 30, 'conjecture_holds': 30}
TRIAL: {'metric_name': 'min_local_index', 'metric_value': 3.272141671993166, 'instances_tested': 30, 'conjecture_holds': 30}
TRIAL: {'metric_name': 'min_local_index', 'metric_value': 3.19592257226448, 'instances_tested': 30, 'conjecture_holds': 30}
TRIAL: {'metric_name': 'min_local_index', 'metric_value': 3.2163421427799777, 'instances_tested': 30, 'conjecture_holds': 30}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_efc79efc.py", line 104, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_efc79efc.py", line 104, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The conjecture was falsified by a counterexample with $n=5$ where $\min_local_index(CS(P)) = 2.321928094887362$, which exceeds the falsification condition | next: Investigate further to understand why the conjecture does not hold for this counterexample and explore alternative hypotheses or corrections to the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13608
2	propose	ollama_remote	glm4:latest	0	0	11667
3	preregistration	ollama_remote	glm4:latest	0	0	6476
4	novelty	ollama_remote	glm4:latest	0	0	4668
5	novelty	ollama_remote	glm4:latest	0	0	5808
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12103
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11868
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10427
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10492
10	judge	ollama_remote	glm4:latest	0	0	11044

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98161 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b63cc36f09d9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b63cc36f09d9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b63cc36f09d9.tar.gz` (if generated)